

# Random number generator

---

## Project proposal

In this project I am going to create a simple web app for the purpose of randomly generating a list of a specified number of numbers. There will also be options for specifying a minimum and maximum value.

## Project design

I used figma to design and prototype my project. This involved creating a basic layout of the app including 3 different inputs and a button to generate the numbers. I used figma prototyping to show how once the button was clicked the numbers generated would appear below.

### Random number generator

How many numbers would you like to generate?

5

The minimum

0

The maximum

10

▶ Generate

# Random number generator

How many numbers would you like to generate?

The minimum

The maximum

▶ Generate

1

6

9

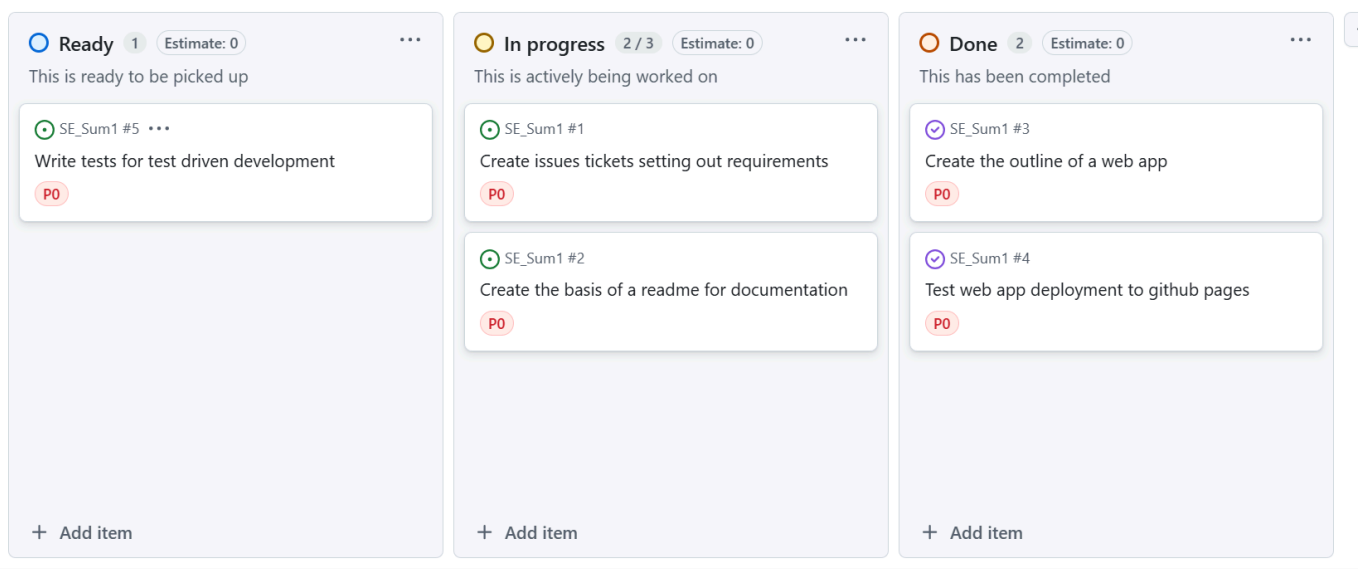
10

3

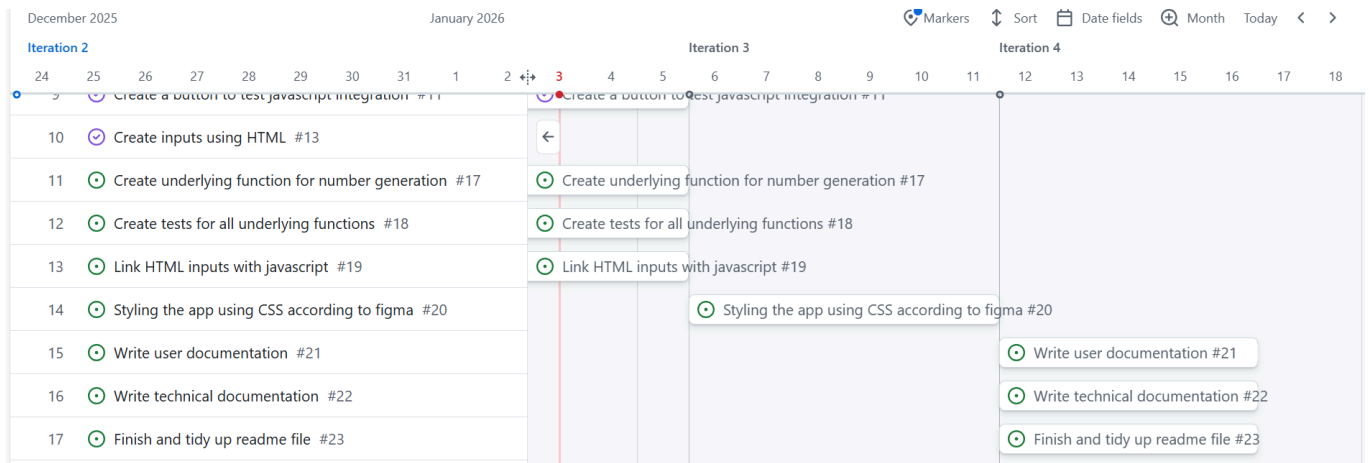
The final project differed slightly from this design.

## Project plan

I used github projects along with issues to plan and manage my project.



To do this I used an agile framework based around sprints, this was because due to holidays I needed to be flexible with project delivery. Working iteratively also ensured that I was able to continuously test the code and deployment with github pages ensuring that errors were spotted early on. I ensured I had a basic plan for each sprint by following a timeline as can be seen below, however, I added additional issues and tasks during each sprint when problems arose.



When writing the tests to check the random integer generator functionality the CI testing which I set-up earlier raised an error in the tests, thankfully this was easily fixable at this stage and I addressed the error in the current sprint by raising additional tickets and reprioritising other tickets accordingly.

```
Run npm test
8 PASS ./functions.test.js
9 FAIL ./randomint.test.js
10   • randomInt works with min greater than max
11
12     expect(received).toBeGreaterThanOrEqual(expected)
13
14     Expected: >= 10
15     Received:    5
16
17     36 |     const max = 1;
18     37 |     const result = randomInt(min, max);
19   > 38 |     expect(result).toBeGreaterThanOrEqual(min);
20       |                   ^
21     39 |     expect(result).toBeLessThanOrEqual(max);
22     40 |   });
23
24     at Object.toBeGreaterThanOrEqual (randomint.test.js:38:20)
25
26 Test Suites: 1 failed, 1 passed, 2 total
27 Tests:      1 failed, 6 passed, 7 total
28 Snapshots:  0 total
29 Time:       0.612 s
30 Ran all test suites.
31 Error: Process completed with exit code 1.
```

## Requirements

I used github issues to set out the requirements of the project using github projects to label the different prioritises so it was easy to see the relative importance of meeting each requirement in the sprint. There are additional requirements which are left as open tickets which could be completed in a second stage of the project using additional sprints.

Once the requirements for the project had been satisfied I merged that branch into the main branch using a pull request. I ensured that the issue was linked to the pull request by tagging it, however, noticed that this often resulted in duplications of an issue as it existed as a pull request and issue in the github project. Hence, I altered the github projects workflow to only add issues rather than pull requests to the project to fix this issue.

## Minimal Viable Product

The first stage of creating the minimal viable project (MVP) was first to understand the tech stack. Having never worked with github pages before I first wanted to ensure that my project could be hosted on the platform. I did this by first creating an HTML file and configuring the github actions to upload this to github pages. I then incorporated css into the HTML file and pushed the updates. The github actions enabled each new update to the branch to automatically update the github pages reducing the manual processes. I then tested the incorporation of Javascript using a basic button and found this successful on the deployed website.

I then focused on writing my unit testing framework to catch any errors during the development phase. This reduces the costs of bug fixes later on in the development. To do this I created a basic functions file with a simple test using the jest testing framework.

After this I expanded the HTML to contain all the inputs and outputs which would be needed in the final application. After doing this I linked the javascript button with the generate button and tested printing a simple text string as the output. I then tested this locally using Live Server where I noticed that the button was not working and a string would not be printed. Upon doing further research I noticed that to import functions into a javascript file for web app you needed to use ES module syntax rather than CommonJS syntax. Once I fixed this the web app ran correctly, however, the tests stopped running as the Jest testing framework I used worked only on CommonJS syntax. To address this I installed and setup the Babel config file, which explicitly calls for ES module syntax to be converted to CommonJS so that the Jest testing runs correctly.

After testing all the HTML and Javascript manually I then added the CSS styling, I referenced the figma design I created earlier while making some tweaks to make the application more user friendly.

## Testing

### Continuous integration testing

I started with a very basic test framework to ensure that jest was working correctly on my system.

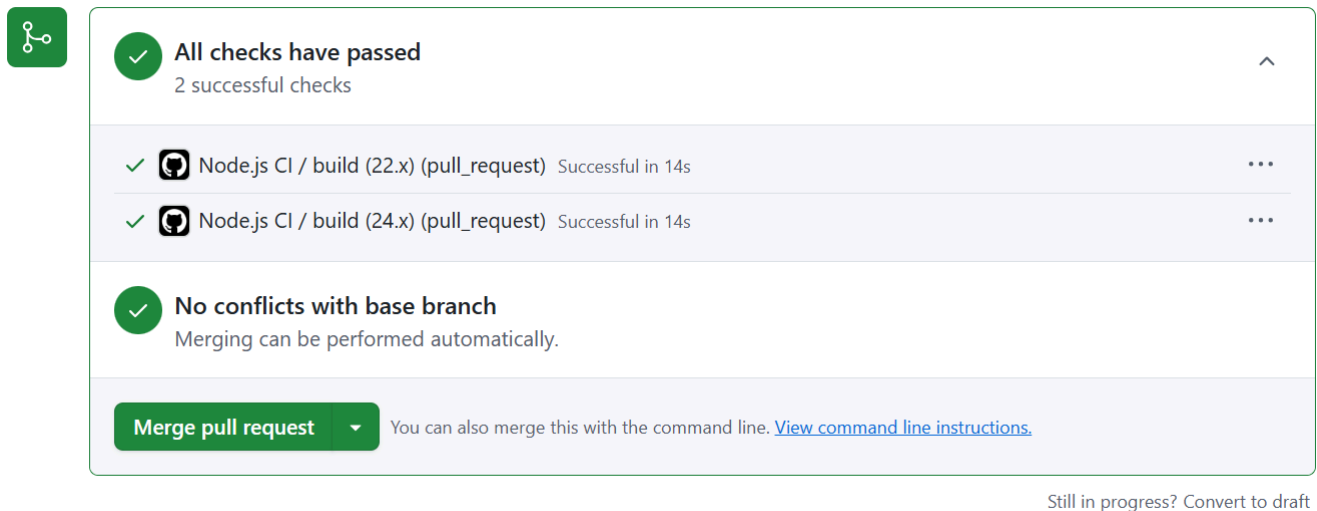
```
C:\Users\hbarn\Documents\University\SE_Sum1\SE_Sum1>npm test

> se_sum1@1.0.0 test
> jest

PASS ./functions.test.js
  ✓ adds 1 + 2 to equal 3 (43 ms)
  ✓ adds 2 + 2 to not equal 3 (2 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        1.738 s
Ran all test suites.
```

Once I had ensured that the tests were working correctly locally I pushed them to github. I then used github actions to setup continuous integration testing. This means that whenever I push changes or do a pull request to the main branch the tests are automatically conducted ensuring that error are picked up immediately. This saves time running tests manually and ensures that high code standards are met.



The image shows a GitHub Actions status bar for a pull request. It features a green checkmark icon on the left. The main status is "All checks have passed" with "2 successful checks" below it. Below this, two specific checks are listed: "Node.js CI / build (22.x) (pull\_request)" and "Node.js CI / build (24.x) (pull\_request)", both marked as "Successful in 14s". A third check, "No conflicts with base branch", is also shown with the note "Merging can be performed automatically." At the bottom, there is a green button labeled "Merge pull request" and a link to "View command line instructions." The bottom right corner of the bar contains the text "Still in progress? [Convert to draft](#)".

✓ All checks have passed  
2 successful checks

✓ Node.js CI / build (22.x) (pull\_request) Successful in 14s

✓ Node.js CI / build (24.x) (pull\_request) Successful in 14s

✓ No conflicts with base branch  
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

## Manual testing

I also conducted manual testing using the web app to ensure that users would find the app easy to use and not encounter any bugs. While doing this I noticed that it was hard to tell when the button was hovered over as it didn't change, hence I added additional CSS styling to address this issue. All other parts of the application worked as expected.

## Accessibility testing

I used the issues tab to inspect the hosted webpage on github pages to view any accessibility concerns. This showed an issue of the HTML element not having a language attribute. This poses an issue for those with screen readers which might be unsure which language the webpage is in.

The screenshot shows the Chrome DevTools 'Issues' panel. At the top, there's a toolbar with icons for chat, copy, print, home, source, console, accessibility, network, performance, and a search icon. Below the toolbar, there are filters: 'Include third-party issues' (checked), 'Severity: Default levels', 'Browser: Top browsers', and 'Group by kind'. The 'Accessibility' section is expanded, showing a warning: '<html> element must have a lang attribute: The <html> element does not have a lang attribute'. A yellow 'Warning' badge is next to it. Below the warning, there's a section titled 'AFFECTED RESOURCES' with a text box containing '<html>'. Below this, there's a link 'Open in Elements'. To the left of the 'FURTHER READING' section, there's a 'REQUEST' section with the URL 'https://hbarnes163.github.io/SE\_Sum1/'. The 'FURTHER READING' section contains three links: 'Learn more about this axe rule at Deque University', 'Learn why this is important and how to fix it on webhint.io', and 'Learn why this is important and how to fix it on webhint.io'. Below the 'Accessibility' section, the 'Other' section is expanded, showing a warning: 'Page layout may be unexpected due to Quirks Mode', also with a yellow 'Warning' badge.

Issues

Include third-party issues Severity: Default levels Browser: Top browsers Group by kind

▼ Accessibility

1 <html> element must have a lang attribute: The <html> element does not have a lang attribute Warning

AFFECTED RESOURCES

<html>

Open in [Elements](#)

REQUEST

[https://hbarnes163.github.io/SE\\_Sum1/](https://hbarnes163.github.io/SE_Sum1/)

FURTHER READING

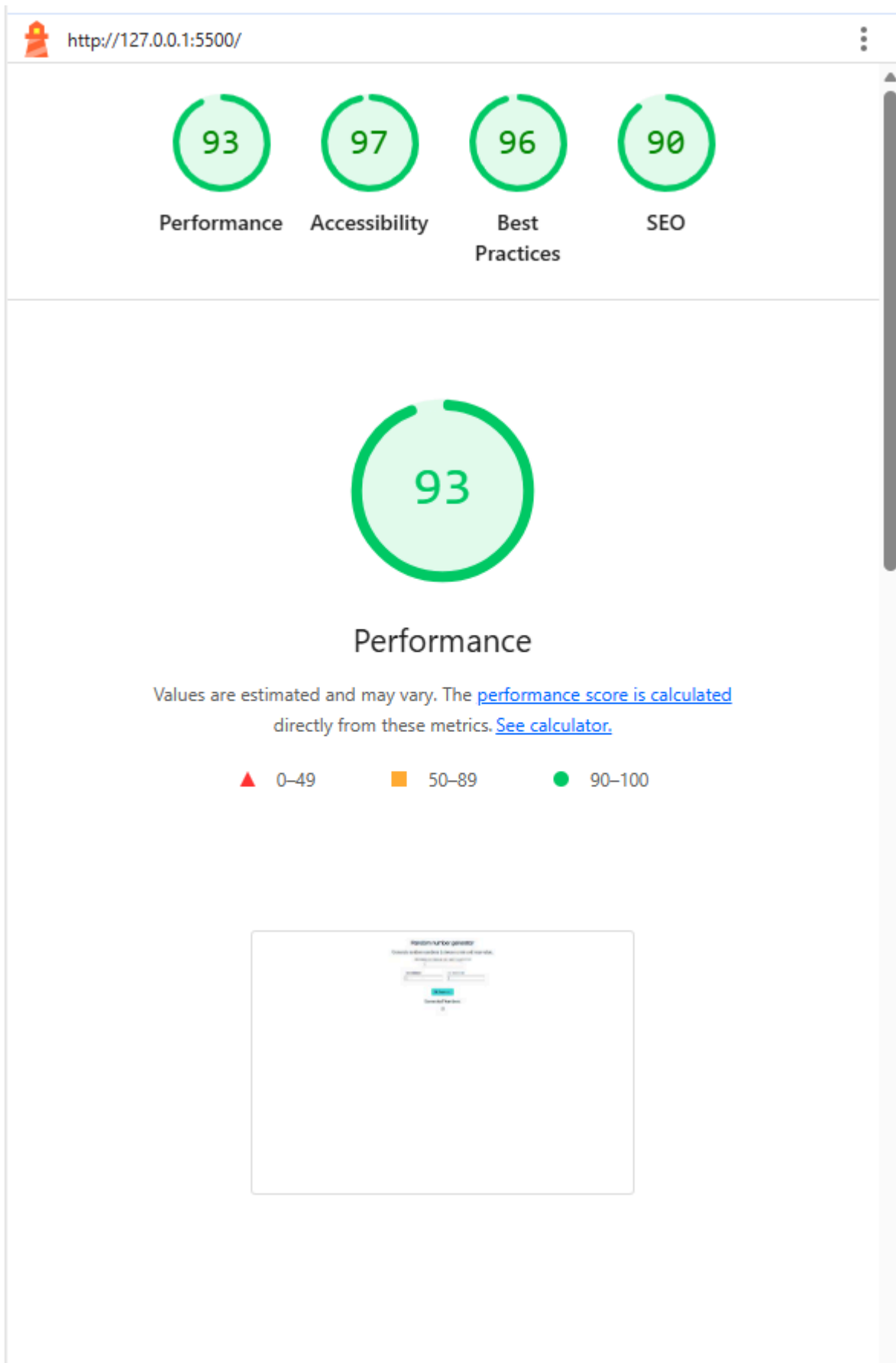
[Learn more about this axe rule at Deque University](#)

[Learn why this is important and how to fix it on webhint.io](#)

▼ Other

1 Page layout may be unexpected due to Quirks Mode Warning

I also used google lighthouse to test the webpage. This resulted in a very good score across all tested attributes including accessibility which scored a 97.



## User documentation

### Using the website

The application can be accessed from the following [website](#). On the website there are 3 different numeric inputs.

**How many numbers do you want to generate?** takes a positive integer input for the number of numbers to generate **The minimum** and **The maximum** each take integer inputs for the minimum and maximum limits of the generated values.

**Generate**, once you are happy with the inputs click on the generate button to generate the list of numbers. If any of the above conditions are violated, e.g -1 numbers to generate, then the generate button will produce no numbers.

## Suggesting feedback and improvements

If you notice a bug or wish to request a new feature you can do so by adding an issue to the [GitHub page](#). Before doing this please look through the open tickets to see if there is already a ticket with your requirements or setting out the bug you have observed. If you feel that the ticket is relevant but doesn't address the full issue then you can set out additional detail in the comments section of that ticket rather than raising a new one.

If there is no relevant ticket then you should create a new issue. Include as much detail as possible in the description of the ticket and tag it with the relevant label. The software engineering team will then be able to look over all open tickets when planning the next sprint and may be in contact with additional queries when they begin to work on the item.

## Technical documentation

### Running the application locally

On organisation non-developer devices it is not possible to install node.js without obtaining additional permissions which require a lengthy approval process. To run the application locally you will either need a developer laptop or to use a personal laptop. If you believe there is business justification for having access to node.js then I recommend reaching out to the analytics enablement hub or the data community who will be able to offer additional advice. You can use Live Server to run the whole application on an organisation laptop and run node in the browser but this won't give you the full functionality.

### Installing node.js and npm

To run this project locally you need an installation of node.js and npm. The easiest way to do this is to install node.js from [this link](#). I downloaded the prebuild version for windows as I didn't have docker on my computer which also came with npm reducing the need for two installations and ensuring compatibility between the two programs.

### Setting up your local environment

To access the code and run the project locally you first need to clone this git repository, to do this you will need to have the software git on your local system. The instructions needed to install git can be found [here](#), once you've downloaded the file then you can click to start the installer and go through the options to configure windows for your device. Once you have git setup then navigate to your preferred IDE, I would recommend using Visual Studio Code as it works well across programming languages and has useful extensions for debugging and running code. If you haven't got Visual Studio Code installed already then the installation guide can be found [here](#).



Once you've opened visual studio there should be a welcome page which appears upon start up, one of the options here is to clone a git repository. Click on that and paste the repository URL which can be found clicking on the Code button in the top right of this Webpage. You should also create a new folder for this code to be stored in. Once you've cloned the repository all the files should appear in the file explorer tab.

To install the correct dependencies open up a new terminal with command prompt, then run `npm install`. This will ensure that all the correct dependencies and versions are installed and you are ready to be able to run the code locally.

### **Using live server to run the application locally**

If working in visual studio code, which I recommend due to its ability to work with JS, HTML and CSS at the same time, you can install the Live Server extension from Ritwick Dey which can be found in the extensions tab in Visual Studio Code. Once installed you can use the Go Live button to load the website in your default browser. This enables live testing of the web app to ensure functionality before it is deployed to github pages.

### **Adding additional features**

If while working on the code you notice that there are additional features or bug fixes which could improve the application then please feel empowered to add to the application. In order to conform to best practice these are the steps which should be taken before additional items are added.

### **Create a new ticket**

You must create a new ticket which is linked to the project as a GitHub issue, or assign yourself to an existing ticket. You should then create a new sprint in the github projects page detailing how long you expect to spend on the code improvement, or multiple improvements if it's a longer sprint or the issues are smaller. Then assign all of the relevant tickets to the sprint. Once this has been done you are ready to begin coding up the improvement

### **Create a new branch**

You must create a new branch to work from, this should have an intuitive name and explain the feature you are aiming to improve. Once you've made the improvements then use Live server to test that the app works correctly on your local device. You must ensure that any new functions have quality tests written and check that all tests work correctly by running `npm test` in the command line. Ensure that the code is commented and update the User and Technical documentation with any changes made. You should also manually test the application and go through UAT testing to address any mistakes before using Live Server and google Lighthouse to check for any accessibility or performance issues.

### **Update the main code**

Once all this is done you can push your code to GitHub and create a pull request. Ensure that you reference which issues are closed in the description of the Pull Request and assign someone to review the changes to the code. Once they are content with the changes and provided the CI testing has passed the Pull Request may be merged into the main branch. This automatically triggers the deployment of the GitHub pages app which will update the Live App with your changes.

## **Evaluation**

Overall, while the design and hence MVP were simple the execution of the project followed good project management standards. The use of GitHub projects with automated actions from GitHub issues made it easy to plan the project ensuring that it remained on track. The MVP has been thoroughly tested and any bugs picked up in development have been resolved. However, there is still room for improvement and for new features to be added. For future sprints developers should refer to the open GitHub issues page which contains new features and bugs to be addressed. Prioritisation and project planning can then be done with the linked GitHub projects page.

I did encounter some issues while working on this project, Jest and Live Server used different syntaxes to import functions from other files resulting in errors which took some time to understand and resolve. The device I initially started working on was not able to install Node meaning I had to setup Git and Node on a different device to work on the development. These issues resulted in certain features being delayed by a sprint but didn't have an effect on the final project delivery which was a success.